

EM-FDR-Bagging: Robust Estimation of the Connectivity Matrix from Multi-State Spiking Data

A bagged, scramble-calibrated wrapper around PRISM-EM

Jan Balewski, NERSC

July 6, 2026

Abstract

This document describes **EM-FDR-Bagging**, a statistical wrapper around the PRISM-EM fitter that aims to recover the shared connectivity matrix $A \in \mathbb{R}^{N \times N}$ of a non-stationary, multi-state Poisson GLM *robustly*, i.e. with explicit control of false edges and with per-edge uncertainty estimates, on real data where no ground truth exists. The method combines three ideas. First, one ordinary PRISM-EM run is made on the requested time range to discover a reference hard Viterbi state sequence. Second, *reference-labeled pair bagging*: every FDR bag draws the same fraction of lag pairs $(\hat{S}_t, Y_{t-1}, Y_t)$ from that reference window and re-estimates only A and $\mathbf{B}$ with the state labels locked. No bag performs a new E-step. Third, a *per-neuron time-shuffle null* and *per-source pooled significance with cross-bag stability selection*: inside each bag, independently time-scrambled neurons are refit by the same locked M-step, edges are tested against a source-column pooled null, and retained only if they recur reliably across bags. The procedure yields, for every surviving edge, a selection frequency, a bagged point estimate, a bootstrap SD, and a null-referenced effect size. The resulting edge masks are expected to be reliable, but the corresponding fitted magnitudes should be interpreted as conservative: the locked M-step still uses L_1 shrinkage and may also be scaled by spectral-radius enforcement. Therefore these outputs are best treated as Stage (b) support estimates, and a follow-up de-biased refit is needed for final magnitudes; see the companion writeup `deBiasFit_states_ver3c.tex` for details. We treat PRISM-EM as a black box exposed through two entry points and specify everything needed for an implementer who has the PRISM-EM code in hand.

Contents

1	Scope and Inputs	3
2	The PRISM-EM Workhorse and Its Two Entry Points	3
2.1	What PRISM-EM fits	4
2.2	Entry point 1: full fit	4
2.3	Entry point 2: M-step-only refit with locked state sequence	4
3	Method Overview	5
4	Stage (a.1): Reference-Labeled Pair Bagging	5
5	Stage (a.2): Real Fit and the Circular-Shift Null	7
5.1	Real locked M-step on the selected pairs	7
5.2	The null: per-neuron independent circular shifts	7
5.3	Null re-estimation through the locked M-step	8
5.4	How many scrambles are needed	8
6	Stage (a.3): Per-Bag Outputs	9

7	Stage (b): Aggregation, Significance, and Pruning	9
7.1	Per-source pooled null and per-bag selection	9
7.2	Cross-bag stability selection	10
8	Per-Edge Uncertainty: Two Distinct Errors	11
8.1	Null spread and effect size	11
8.2	Estimation variability: the across-bag bootstrap SD	12
8.3	Reported per-edge record	13
9	Implementation and Runtime Arguments	13
9.1	<code>prism_FDR_Bags_train3c.py</code> : Stage (a)	13
9.2	<code>prism_EM_FDR_Bags_aggregate3c.py</code> : Stage (b)	16
10	End-to-End Summary	18

1 Scope and Inputs

The full analysis pipeline is organized into three stages. **Stage (a)** is the ordinary PRISM-EM fit on the chosen spike-train window: it estimates a shared connectivity matrix, state-dependent biases, and a reference decoded state sequence; this core fitter is specified in `fitEM.states_ver3c.tex`. **Stage (b)** is the EM-FDR-Bagging procedure described in this document: it reuses the Stage (a) reference state labels, performs bagged locked M-step refits, calibrates edges against time-scrambled null fits, and returns a statistically vetted support mask with conservative edge magnitudes. **Stage (c)** is the follow-up de-biased refit specified in `deBiasFit.states_ver3c.tex`: it freezes the Stage (b) support and Dale-sign decisions, removes the L_1 sparsity penalty, and refits final edge magnitudes and state biases under the remaining biological and stability constraints.

The input is a single spike-train recording of N neurons over T discrete time bins of width Δt , in the format consumed by PRISM-EM (`spikesData/<dataName>.spikes.npz`). The target regime, which sizes every numerical choice below, is: $N = 50$ – 500 neurons; recordings of 10–60 minutes at $\Delta t = 10$ ms, so T ranges from 6×10^4 to 3.6×10^5 bins; per-neuron firing rates of 1–5 Hz; state-switching rates of 5–15 transitions per minute; and a small number of latent states, typically $M = 2$. Crucially, the method is designed for application to *real* data: it never references ground truth, and every quantity it reports is computed from the data alone.

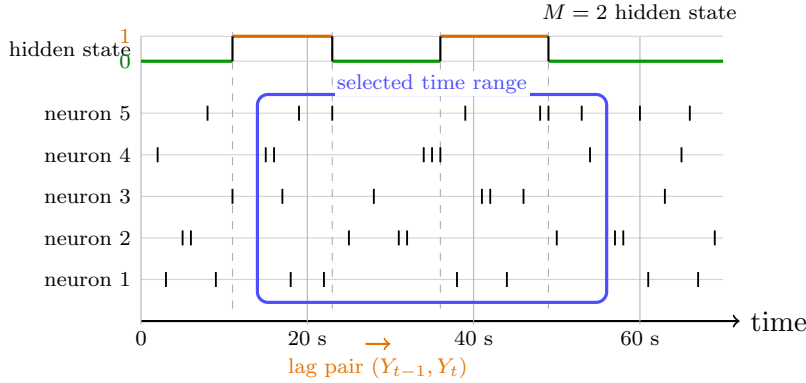


Figure 1: Cartoon of the spike-train input. Rows are neurons, vertical ticks are spike events in discrete time bins, and the colored track above the raster sketches a hidden $M = 2$ state process switching every few seconds. The blue box marks the reference time window supplied to the initial PRISM-EM fit. FDR bags later sample lagged pairs $(\hat{S}_t, Y_{t-1}, Y_t)$ from this same window.

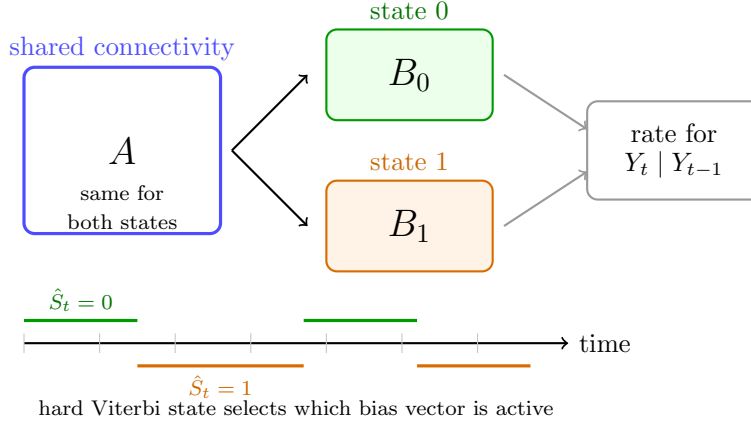
The deliverable is not a final unregularized point estimate of A but a vetted edge list and support mask. For each off-diagonal entry (i, j) that survives the procedure we report a point estimate, a statistical uncertainty, a stability (selection) frequency, and an effect size relative to the null. These values are useful for deciding which edges exist, but the edge magnitudes are likely underestimated because of the same L_1 shrinkage and spectral-radius projection used during fitting. Final magnitude recovery is handled by the Stage (c) de-biased refit described in `deBiasFit.states_ver3c.tex`. Self-couplings A_{ii} are excluded from testing throughout.

2 The PRISM-EM Workhorse and Its Two Entry Points

EM-FDR-Bagging treats PRISM-EM as a callable. We summarize only what the wrapper relies on; the full model, objective, and defaults are documented separately in the PRISM-EM specification.

2.1 What PRISM-EM fits

PRISM-EM fits a non-stationary Poisson GLM in which all M latent states share one connectivity matrix A but carry state-specific bias vectors B_m . With c_t the simplex coefficients at bin t and $B_t = \sum_m c_{mt} B_m$, the rate is $\mu_t = \exp(\min(AY_{t-1} + B_t, \eta_{\text{clip}})) \Delta t$ and $Y_t \sim \text{Poisson}(\mu_t)$. Training is block coordinate descent: an E-step infers $c_{1:T}$ by simplex-projected gradient descent and Viterbi-decodes them to a hard one-hot state sequence $\hat{S}_{1:T}$; an M-step then updates $A, \mathbf{B}$ by Adam under the frozen one-hot assignment, with off-diagonal L_1 shrinkage (strength λ_3), a spectral-radius projection $\rho(A) \leq \rho_{\text{max}}$, and a learning-rate decay. The shrinkage, projection, and decay each switch on only after a configurable number of EM iterations. The fitted A is therefore the output of a specific, deterministic-in-configuration pipeline of regression plus shrinkage plus spectral projection — a fact the null construction below depends on.



$$\eta_t = AY_{t-1} + B_{\hat{S}_t}, \quad \tilde{\eta}_t = \min(\eta_t, \eta_{\text{clip}}), \quad \lambda_t = \exp(\tilde{\eta}_t) \Delta t, \quad Y_t \sim \text{Poisson}(\lambda_t).$$

Figure 2: Two-state PRISM-EM cartoon. This is the EM fitting model, not the synthetic data generator. The lagged spike vector Y_{t-1} always enters through the same shared connectivity matrix A . The decoded state $\hat{S}_t \in \{0, 1\}$ selects the active bias vector, B_0 or B_1 , for the conditional rate of Y_t . FDR bagging later locks this state sequence and refits only A and the two bias vectors.

2.2 Entry point 1: full fit

$$\text{full_fit}(\text{spikes}, M, \theta_{\text{EM}}) \longrightarrow (\hat{A}, \hat{S}_{1:T}, \hat{\mathbf{B}}, \text{history})$$

This is the ordinary PRISM-EM run: full E/M alternation on the supplied spikes, returning the fitted connectivity $\hat{A}$, the decoded one-hot state sequence $\hat{S}_{1:T}$, the per-state biases $\hat{\mathbf{B}}$, and the training history. Here θ_{EM} collects all PRISM-EM hyperparameters (number of EM iterations, M-step epochs, learning rates, $\lambda_2, \lambda_3, \rho_{\text{max}}$, the delay schedules, etc.).

2.3 Entry point 2: M-step-only refit with locked state sequence

$$\text{mstep_locked}(Y_{\text{prev}}, Y_{\text{curr}}, \hat{S}^{\text{lock}}, A^{\text{init}}, \hat{\mathbf{B}}^{\text{init}}, \theta_M) \longrightarrow (A, \mathbf{B})$$

This entry point runs *only* the M-step. It accepts spike pairs $(Y_{\text{prev}}, Y_{\text{curr}})$, an externally supplied and *frozen* state sequence $\hat{S}^{\text{lock}}$ (used directly as the one-hot assignment, with no E-step and no Viterbi re-decoding), initial parameters $(A^{\text{init}}, \hat{\mathbf{B}}^{\text{init}})$, and returns the re-estimated connectivity A and the refit state biases $\mathbf{B}$. In the null workflow the initial A and $\mathbf{B}$ are not copied from the real locked fit. Instead, for every scramble they are initialized freshly from the scrambled data, or randomly, according to `null_A_init` and

`null_B_init`. Re-fitting $\mathbf{B}$ in the scrambled context lets the null absorb state-specific baseline effects, so the test of A is not made artificially harsh by a bias mismatch; fresh A initialization avoids seeding the null from the real edge pattern.

Locked-state time indexing. For each lag-1 training pair (Y_{t-1}, Y_t) , the locked state used by the M-step is $\hat{S}_t^{\text{lock}}$, i.e. the state attached to the destination or current bin whose conditional rate is being modeled. Equivalently, if the spike pairs have length $T - 1$, the state covariate rows are formed from $\hat{S}_{1:T-1}^{\text{lock}}$, converted to one-hot vectors. This convention is part of the FDR bagging definition and must be used identically for every real and scrambled locked refit.

The locked M-step is configured to reproduce the real pipeline exactly: the same L_1 strength λ_3 , the same spectral projection $\rho_{\max}$, and the same learning-rate decay as the corresponding `full_fit`. Because there is no E-step here, the EM iteration counters that gate PRISM-EM’s delayed schedules are not meaningful; instead all schedule-gated operations (shrinkage, spectral projection, decay) are **active from the first M-step epoch**. The rationale is that the null estimate must traverse the same shrinkage and constraint geometry as the converged real estimate, which lives in the late-EM regime where all schedules have already fired.

Implementation note. In the current `ver3c` code these two entry points live in `PrismEM_Workhorse3c.py`: `run_full_fit` and `run_locked_mstep`. The locked M-step consumes externally supplied spike pairs, an externally supplied $\hat{S}^{\text{lock}}$, and explicit initial A and $\mathbf{B}$. Its lag-1 state alignment for the new FDR bagging path must be $\hat{S}_t^{\text{lock}}$ for pair (Y_{t-1}, Y_t) . If existing helper names still refer to “previous states”, their use must be audited while implementing this plan.

3 Method Overview

EM-FDR-Bagging has two stages, and the document is organized to match the intended software structure.

Reference EM fit (performed before Stage (a)). Run `prism_EM_train3c.py` once on the desired `--time_range_sec`. This discovers the reference state sequence $\hat{S}_{0:T_{\text{ref}}-1}^{\text{ref}}$ and writes a tagged reference fit, for example `prismFit/<dataName>_jXXXX.prismEM.npz`. Its EM hyperparameters control state discovery; once the reference state labels are good enough, the final precision of A and $\mathbf{B}$ is delegated to the locked bag fits.

Stage (a), per-bag fitting (Sections 4–6). For a single bag: load the reference EM fit and original spikes, draw a fixed-size random subset of reference-labeled lag pairs $(\hat{S}_t^{\text{ref}}, Y_{t-1}, Y_t)$, re-estimate only $(\hat{A}^{(b)}, \hat{\mathbf{B}}^{(b)})$ by `mstep_locked`, then run P independently time-scrambled locked-M-step null refits on the same selected pair indices. Write everything to disk. This stage is one self-contained program; it is run N_{bag} times, once per bag, as independent jobs. The bag trainer has its own locked-M-step training hyperparameters. Their defaults are read from the reference EM metadata for continuity, but they may be overridden on the Stage (a) command line and the effective values are saved in every bag file.

Stage (b), aggregation and pruning (Section 7). Read all per-bag outputs from a directory, build per-source pooled null distributions, perform per-bag selection and cross-bag stability selection, and emit the final vetted edge list with uncertainties.

The symbols introduced for the wrapper, kept distinct from PRISM-EM’s N, M, T , are collected in Table 1.

4 Stage (a.1): Reference-Labeled Pair Bagging

A reference EM fit is produced before any bag job is launched:

`prism_EM_train3c.py` on `--time_range_sec = [t0, t1]`.

Symbol	Meaning	Nominal value
T_{ref}	bins in the reference EM time window	from reference fit
N_{pair}	lag pairs in the reference window, $T_{\text{ref}} - 1$	from reference fit
f_{bag}	pair fraction per bag, <code>--bag_frac</code>	0.8
K_{bag}	pairs sampled per bag, $\lfloor f_{\text{bag}} N_{\text{pair}} \rfloor$	from <code>--bag_frac</code>
N_{bag}	number of bags	5–10
P	scrambles per bag	4–8
$\delta_{\text{roll}}^{\text{min}}$	roll exclusion-zone half-width (s)	2
q	per-bag per-source selection quantile level	e.g. 0.99
π_{thr}	cross-bag selection-frequency threshold	0.6–0.8
M, N, T	states, neurons, bins (PRISM-EM)	≈ 3 ; 50–500; 6×10^4 – 3.6×10^5

Table 1: Symbols introduced by EM-FDR-Bagging and their nominal values. Every numeric choice in the text is written as “symbol = value” so it can be adjusted.

The reference output fixes the training window, the fitted soft state coefficients $\hat{c}^{\text{ref}}$, and the hard Viterbi labels $\hat{S}_{0:T_{\text{ref}}-1}^{\text{ref}}$. The bag driver does not accept a new `--time_range_sec`; it inherits the reference fit’s saved window and therefore uses the same source data for simulations and experiments.

From the reference window we form a table of lag pairs

$$X_t = Y_{t-1}, \quad Z_t = Y_t, \quad H_t = \hat{S}_t^{\text{ref}}, \quad t = 1, \dots, T_{\text{ref}} - 1.$$

Here X_t is the source/history spike vector, Z_t is the destination/current spike vector, and H_t is the destination-bin state label that controls the conditional rate for Z_t . This table contains $N_{\text{pair}} = T_{\text{ref}} - 1$ rows.

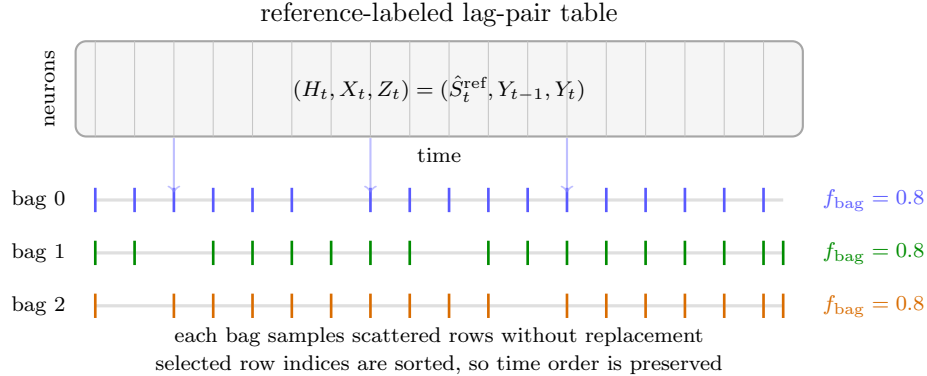


Figure 3: Reference-labeled pair bagging. The initial EM fit defines one time window and one table of lagged rows $(\hat{S}_t^{\text{ref}}, Y_{t-1}, Y_t)$. Each FDR bag samples a different random 80% subset of rows without replacement; the selected rows are sparse but kept in chronological order. The same selected row indices are used for all neurons inside a bag, so real cross-neuron synchrony is preserved.

For bag b , draw an index set $I_b \subset \{1, \dots, N_{\text{pair}}\}$ without replacement, with

$$|I_b| = K_{\text{bag}} = \lfloor f_{\text{bag}} N_{\text{pair}} \rfloor, \quad f_{\text{bag}} = \text{--bag_frac}.$$

The same row index set is used for all neurons. This preserves the within-time synchrony between neurons in the real bag fit while giving every bag the same statistical precision. All bag indices are equivalent: `bag_idx=0` is no longer chronological or special; it is simply the first random subset in the reproducible bag-indexed RNG stream.

This design deliberately drops the previous block-concatenation machinery. There are no block starts, no block boundaries, no minimum block-start separation, and no bag-specific E-step. The only perturbation defining a real bag is the random subset of already-labeled lag-pair rows.

5 Stage (a.2): Real Fit and the Circular-Shift Null

5.1 Real locked M-step on the selected pairs

For bag b , run entry point 2 on the selected rows:

$$(\hat{A}^{(b)}, \hat{\mathbf{B}}^{(b)}) = \text{mstep_locked}(X_{I_b}, Z_{I_b}, H_{I_b}, A^{\text{init},(b)}, \mathbf{B}^{\text{init},(b)}, \theta_M).$$

This $\hat{A}^{(b)}$ is the bag’s real connectivity estimate. The state labels are the locked hard labels from the reference EM fit; there is no E-step, no Viterbi decoding, and no re-estimation of $\hat{c}_t$ inside a bag.

5.2 The null: per-neuron independent circular shifts

The null hypothesis for an edge is “no directed lag-1 coupling from neuron j to neuron i beyond what the bias/state structure already explains.” We realize it by destroying cross-neuron temporal alignment while preserving each neuron’s own marginal statistics and the state timeline. Concretely, for scramble $p = 1, \dots, P$ we draw an independent integer circular shift $\delta_n^{(b,p)}$ for each neuron n and roll that neuron’s spike train over the full reference window:

$$\tilde{Y}_{t,n}^{(b,p)} = Y_{(t - \delta_n^{(b,p)}) \bmod T_{\text{ref}}, n}.$$

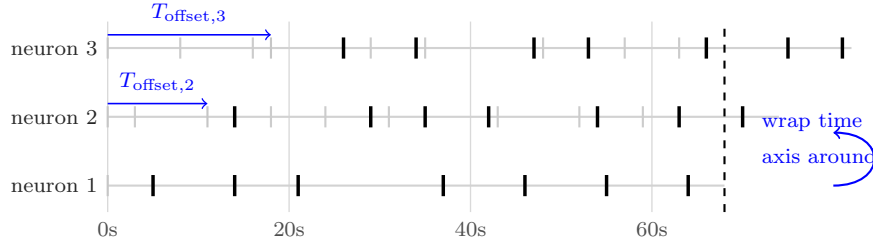


Figure 4: One circular-shift null scramble for one bag. For each neuron, original spike times (grey ticks) are shifted to the right by an independent offset $T_{\text{offset},n}$ to produce the scrambled spike train (black ticks). Spikes that are shifted beyond the end of the window (dashed line) wrap around to the beginning.

Because every neuron is shifted by a different random amount, the precise lag-1 relationship that A captures is broken, while each neuron’s firing rate, burstiness, and count distribution are untouched (a circular roll is a pure permutation of a neuron’s own time axis). The locked reference state labels $H_t = \hat{S}_t^{\text{ref}}$ are *not* rolled: they are held fixed so that the null asks specifically about coupling, not about state structure.

Two-sided exclusion zone on each shift. A shift near 0 leaves the data almost unshifted and a shift near T_b wraps back to almost unshifted; both leave the real lag structure largely intact and would contaminate the null toward the alternative. We therefore restrict each shift to

$$\delta_n^{(b,p)} \in [\delta_{\text{roll}}^{\text{min},\text{bin}}, T_{\text{ref}} - \delta_{\text{roll}}^{\text{min},\text{bin}}], \quad \delta_{\text{roll}}^{\text{min},\text{bin}} = \delta_{\text{roll}}^{\text{min}} / \Delta t,$$

with $\delta_{\text{roll}}^{\text{min}} = 2 \text{ s}$ (200 bins). The exclusion is two-sided by construction: both the small-shift and the near-full-wrap small-effective-shift regions are forbidden.

No pairwise separation among neurons. The shifts of different neurons need not be separated from one another. The null only requires that each neuron’s shift is individually non-trivial (the two-sided zone above) and that shifts are drawn independently across neurons; a coincidence in which two neurons happen to share the same shift is harmless to null validity. This matters in practice: with a 10-minute bag and a 2 s separation there would be only a finite number of effective “slots,” and a bag may contain up to 500 neurons, so any attempt to enforce pairwise distinct shifts would be infeasible by pigeonhole. We deliberately avoid that requirement. The implemented draw routine simply samples independently from the allowed two-sided interval for each neuron.

5.3 Null re-estimation through the locked M-step

For each scramble p we recompute connectivity with the state sequence frozen, via entry point 2:

$$(A^{\text{null},(b,p)}, \mathbf{B}^{\text{null},(b,p)}) = \text{mstep_locked}(\tilde{X}_{I_b}^{(b,p)}, \tilde{Z}_{I_b}^{(b,p)}, H_{I_b}, A^{\text{init},(b,p)}, \mathbf{B}^{\text{init},(b,p)}, \theta_M).$$

The scrambled pair arrays $\tilde{X}_t^{(b,p)} = \tilde{Y}_{t-1}^{(b,p)}$ and $\tilde{Z}_t^{(b,p)} = \tilde{Y}_t^{(b,p)}$ are formed from the rolled reference-window spikes, and the same selected row indices I_b are used as in the real fit. The locked state attached to pair $(\tilde{Y}_{t-1}^{(b,p)}, \tilde{Y}_t^{(b,p)})$ is $H_t = \hat{S}_t^{\text{ref}}$. Only the spike counts entering the null M-step are circularly shifted. The null initialization is fresh for each scramble: $A^{\text{init},(b,p)}$ is either computed from the scrambled spike train (`null_A_init=scrambled_data`, the default) or drawn randomly (`null_A_init=rand`), and $\mathbf{B}^{\text{init},(b,p)}$ is likewise initialized from scrambled data (`null_B_init=scrambled_data`, the default) or randomly (`null_B_init=rand`). The real locked $\hat{A}^{(b)}$ is deliberately not used to seed the null refit.

The real locked fit defaults to data-driven initialization (`init_A=data`, `init_B=data`) rather than reference-fit initialization. This sacrifices some computational convenience, but it makes each bag start from its own sampled lag-pair evidence and therefore preserves more of the sampling variance that the FDR/stability procedure is designed to expose.

The configuration θ_M matches the real locked-M-step pipeline exactly (Section 2.3): identical λ_3 , identical ρ_{max} , identical learning-rate decay, all active from the first epoch. This is the crux of the calibration: the real $\hat{A}^{(b)}$ and the null $A^{\text{null},(b,p)}$ are both outputs of the *same* regression, shrinkage, and spectral projection, applied to the same state timeline, differing only in whether cross-neuron alignment was destroyed and in the resulting refit biases. A significance statement of the form “this edge is larger than the same machinery produces on scrambled data” is therefore meaningful; using a raw, unshrunk M-step for the null would compare against a differently biased quantity and invalidate the test.

5.4 How many scrambles are needed

A useful efficiency follows from testing significance per source column rather than per edge (Section 7). For source neuron j we pool the $(N - 1)$ off-diagonal null weights of that column across all scrambles. A *single* scramble therefore already contributes $\sim (N - 1)$ null samples to source j ’s distribution — of order 199 for $N = 200$. With a modest $P = 4\text{--}8$, the pooled per-source null holds $(N - 1) \times P \approx 800\text{--}1600$ samples, ample to resolve a 1% tail. By contrast a *per-edge* null would need on the order of hundreds of scrambles just to populate a single edge’s distribution, which is infeasible at this budget. The two regimes are not equivalent: the $(N - 1)$ samples within one scramble are not independent (they share a single scrambled realization of the data), so increasing P still helps by adding genuinely independent null realizations and by averaging out scramble-to-scramble variability. The recommendation is to keep P small (4–8) but greater than one, precisely because per-source pooling supplies the raw sample count while extra scrambles supply independence.

6 Stage (a.3): Per-Bag Outputs

Each per-bag job writes a self-contained file to a shared results directory, with a name carrying the bag index so that N_{bag} parallel jobs do not collide. The file contains, at minimum: the real connectivity $\hat{A}^{(b)} \in \mathbb{R}^{N \times N}$; the stack of null connectivities $\{A^{\text{null},(b,p)}\}_{p=1}^P$ of shape (P, N, N) ; the frozen reference state labels used by the selected pairs; the selected pair indices I_b ; `bag_frac`; the reference fit filename and reference time-range metadata; the realized per-neuron shifts (for reproducibility and for the data-reuse study); the stack of refit null biases $\{B^{\text{null},(b,p)}\}_{p=1}^P$ of shape (P, M, N) ; the fitted off-diagonal sparsity of $\hat{A}^{(b)}$ (number and fraction of surviving edges) as a sanity cross-check; and the locked-M-step history summary. Stage (b) consumes only these files and never re-runs PRISM-EM.

Per-bag program output naming. One invocation of `prism.FDR_Bags_train3c.py` produces one bag file containing the real locked-M-step fit and all P scramble refits for that bag. The filename is deterministic and includes `outFitName` and `bag_idx`, for example `<outFitName>.bag003.prismFDRbag.npz`. The real fit fields are written in the same outer schema as the standalone `prism_EM_train3c.py` output where practical, so the usual plotting code can inspect the real part of a bag. Time-series fields used for plotting are copied from the reference EM fit or explicitly marked as reference-derived; they are not newly inferred in the bag job.

Per-bag driver runtime controls. The bag driver uses the locked M-step controls needed to estimate A and B (`m_epochs`, learning rates, $\lambda_3, \rho_{\text{max}}$, schedule controls as applicable, batch size, and initialization modes for the locked real fit). Its additional controls are: `bag_idx`, `outFitName`, `bag_frac`, `num_scrambles`, `min_roll_shift_sec`, `null_A_init`, and `null_B_init`. It does not accept `--time_range_sec`, `--num_blocks`, `--block_len_sec`, `--min_block_start_sep_sec`, or `--max_block_draw_trials`. There is no separate `null_m_epochs` parameter: the locked null M-step inherits the M-step epoch count and optimizer configuration from the locked real fit, with pruning, spectral projection, and learning-rate decay active from the first locked epoch.

Compute layout. Each per-bag job is a single SLURM job on a single node using $4 \times A100$, matching the existing PRISM-EM launch (`torchrun --standalone --nnodes=1 --nproc_per_node=4`). The P scramble refits reuse the same node and the frozen reference labels, so they are cheap relative to a full EM fit. The N_{bag} jobs are mutually independent and submitted in parallel.

7 Stage (b): Aggregation, Significance, and Pruning

Stage (b) is a single postprocessing program pointed at the results directory. It performs per-source null pooling, per-bag selection, and cross-bag stability selection, then reports the vetted edge list with uncertainties.

7.1 Per-source pooled null and per-bag selection

We test significance *per source column*, exploiting the assumption that the outgoing edges from a given source neuron j are exchangeable under the null: they share that source neuron’s spike statistics and shuffled history stream. This matches the source-column convention A_{ij} = effect of source neuron j on target neuron i , and aligns the null threshold with source-neuron quantities such as firing rate, Dale type, outgoing edge count, and `Sedge`. The per-source exchangeability assumption is what makes the small scramble budget viable (Section 5.4).

For bag b and source column j , pool the off-diagonal null weights across all scrambles of that bag,

$$\mathcal{N}_j^{(b)} = \{A_{ij}^{\text{null},(b,p)} : i \neq j, p = 1, \dots, P\}, \quad |\mathcal{N}_j^{(b)}| = (N - 1) P,$$

and form a per-source threshold as the upper- q quantile of the magnitudes,

$$\tau_j^{(b)} = \text{Quantile}_q(\{|x| : x \in \mathcal{N}_j^{(b)}\}), \quad q = 0.99 \text{ (nominal)}.$$

An edge is *selected in bag b* if its real magnitude exceeds the source threshold,

$$\text{sel}_{ij}^{(b)} = \mathbb{I}[|\hat{A}_{ij}^{(b)}| > \tau_j^{(b)}], \quad i \neq j.$$

See Figure 5 for a cartoon illustrating this quantile-based selection process.

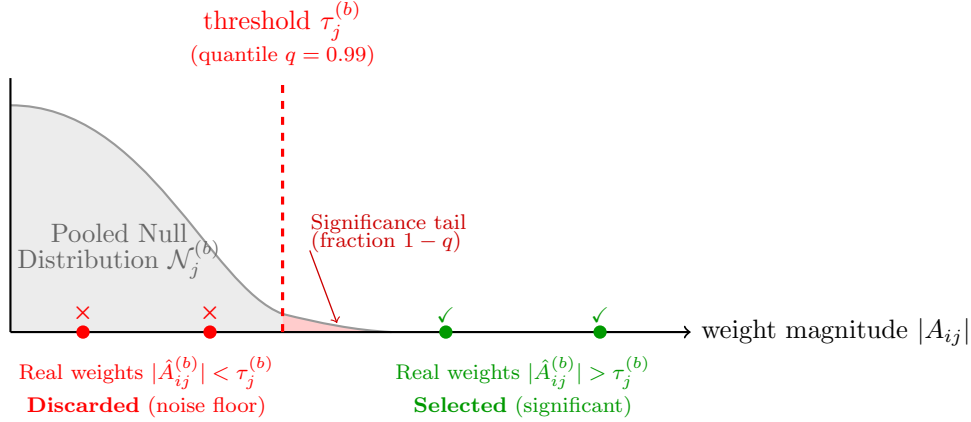


Figure 5: Cartoon of quantile-based edge selection per bag. The magnitude of each real fitted edge weight (dots) is compared against the threshold $\tau_j^{(b)}$ derived from the pooled null distribution of source neuron j . Real weights below the threshold (red $\times$) are discarded as noise; only weights exceeding the threshold (green $\checkmark$) are selected in this bag.

Equivalently, each real edge can be converted to a within-source p -value against the pooled null and the Benjamini–Hochberg procedure applied within the source column; the quantile-threshold form above is the operational default. The expected number of edges selected per bag and per source is recorded as a diagnostic (it is the quantity q_Λ of the stability bound below); it is *measured*, never prescribed, so the algorithm discovers how many edges it keeps rather than being told a target sparsity.

7.2 Cross-bag stability selection

A single bag’s selection is noisy. We retain only edges that are selected reliably across bags. Define the selection frequency

$$\pi_{ij} = \frac{1}{N_{\text{bag}}} \sum_{b=1}^{N_{\text{bag}}} \text{sel}_{ij}^{(b)},$$

and keep edge (i, j) in the final network if $\pi_{ij} \geq \pi_{\text{thr}}$, with $\pi_{\text{thr}} = 0.6$ – 0.8 (nominal). This is Meinshausen–Bühlmann stability selection: the data are perturbed by resampling, the base selector is run on each perturbation, and only consistently chosen variables survive. For example, the implementation default `--stab_sel_thresh 0.7` means that an off-diagonal edge must be selected in at least 70% of the bags to survive. The value assigned to a surviving edge is not weighted by the selection frequency or by any bag quality score; it is the plain arithmetic mean of $\hat{A}_{ij}^{(b)}$ over the bags in which that edge passed the per-bag null threshold. Bags where the edge did not pass the per-bag selector do not enter this final $\hat{A}$ value, although the all-bag mean is also saved as a diagnostic. See Figure 6 for a cartoon of this cross-bag pruning step.

											$\pi_{\text{thr}} = 0.7$	
	bag 0	bag 1	bag 2	bag 3	bag 4	bag 5	bag 6	bag 7	bag 8	bag 9	selection frequency π_{ij}	cross-bag decision
edge (i, j)	✓	✓	✓	✓	✓	✓	✓	✓	✗	✗	8/10 = 0.8	keep
edge (i', j')	✓	✗	✓	✗	✗	✓	✗	✗	✗	✗	3/10 = 0.3	drop

✓

selected in bag b : $|\hat{A}_{ij}^{(b)}| > \tau_j^{(b)}$

✗

not selected in bag b (below per-bag null threshold)

Figure 6: Cartoon of cross-bag stability selection. Each column is one FDR bag; each row is one candidate off-diagonal edge. A green ✓ means the edge passed the per-bag null threshold in that bag (Section 7.1); a red ✗ means it did not. The selection frequency π_{ij} is the fraction of bags with a checkmark. Edge (i, j) appears in 8 of 10 bags ($\pi_{ij} = 0.8 \geq \pi_{\text{thr}}$) and is kept in the final network; edge (i', j') appears in only 3 bags ($\pi_{ij} = 0.3 < \pi_{\text{thr}}$) and is removed.

Reported stability bound. The implementation exposes the pair (q, π_{thr}) : a per-bag per-source quantile level q and a cross-bag frequency threshold π_{thr} . It also reports the Meinshausen–Bühlmann-style stability-selection diagnostic

$$\mathbb{E}[V] \leq \frac{1}{2\pi_{\text{thr}} - 1} \cdot \frac{q_{\Lambda}^2}{p_{\text{cand}}},$$

where $\mathbb{E}[V]$ is the expected number of false edges, q_{Λ} is the *measured* average number of edges selected per bag, and $p_{\text{cand}} = N(N-1)$ is the number of candidate off-diagonal edges. The current program does not derive π_{thr} from a target false-edge count; the user sets `--per_bag_quantile` and `--stab_sel_thresh` directly, and the bound is saved as a diagnostic. The honest caveat is that the bound assumes independent exchangeable bags, whereas all bags are random subsets from the same finite reference-labeled pair table and can overlap in selected rows. They are therefore positively correlated, so the reported value should be read as an approximate stability diagnostic rather than as a guaranteed FDR.

8 Per-Edge Uncertainty: Two Distinct Errors

The design furnishes two genuinely different per-edge “errors,” which answer different questions and must not be conflated.

8.1 Null spread and effect size

For source column j , the pooled null $\mathcal{N}_j = \bigcup_b \mathcal{N}_j^{(b)}$ has a mean and standard deviation,

$$m_j^{\text{null}} = \text{mean}(\mathcal{N}_j), \quad s_j^{\text{null}} = \text{SD}(\mathcal{N}_j),$$

which characterize the *noise floor*: how large an edge weight looks purely by chance once cross-neuron alignment is destroyed and the same machinery is applied. This is not the uncertainty of the estimate; it is the spread of the null. It does, however, define a natural null-referenced effect size,

$$z_{ij} = \frac{\bar{A}_{ij} - m_j^{\text{null}}}{s_j^{\text{null}}},$$

a per-edge signal-to-noise ratio against the scramble null, where $\bar{A}_{ij}$ is the bagged point estimate defined next. See Figure 7 for a cartoon of this null-referenced effect size.

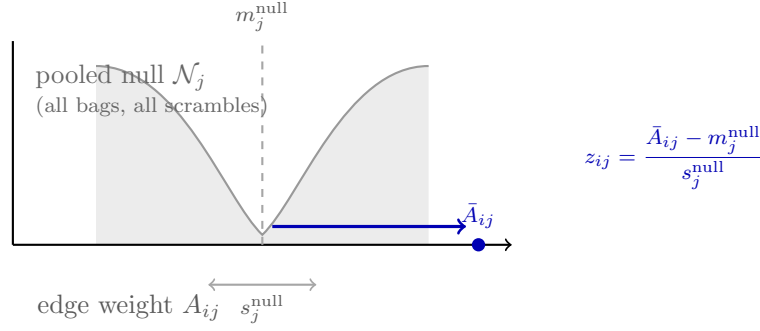


Figure 7: Cartoon of null spread and null-referenced effect size. **Question answered:** how large does this edge look relative to the scramble-noise floor? The gray curve is the pooled per-source null distribution $\mathcal{N}_j$ from all scrambled locked refits. Its mean m_j^{null} and standard deviation s_j^{null} define that noise floor: how large an edge weight can appear by chance once cross-neuron alignment is destroyed. The real bagged estimate $\bar{A}_{ij}$ (blue dot) is compared to the floor through z_{ij} , a signal-to-noise ratio against the null. This is *not* the sampling error of $\bar{A}_{ij}$; that distinct quantity is the across-bag bootstrap SD (Section 8.2).

8.2 Estimation variability: the across-bag bootstrap SD

The quantity one usually means by “the statistical error of an edge” is the sampling variability of its fitted weight, and bagging estimates it directly. Each bag yields a real fitted weight $\hat{A}_{ij}^{(b)}$ on a different random subset of the reference-labeled lag pairs; their spread across bags is a pair-subsampling estimate of the estimator’s sampling distribution. We report the bagged mean and bootstrap SD,

$$\bar{A}_{ij} = \frac{1}{B_{ij}} \sum_{b \in \mathcal{B}_{ij}} \hat{A}_{ij}^{(b)}, \quad \widehat{\text{SD}}_{ij} = \sqrt{\frac{1}{B_{ij} - 1} \sum_{b \in \mathcal{B}_{ij}} (\hat{A}_{ij}^{(b)} - \bar{A}_{ij})^2},$$

where $\mathcal{B}_{ij}$ is the set of bags entering the average and $B_{ij} = |\mathcal{B}_{ij}|$. See Figure 8 for a cartoon of this across-bag estimation variability.

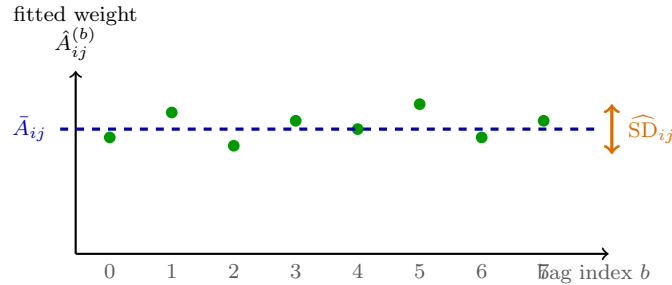


Figure 8: Cartoon of across-bag bootstrap SD. **Question answered:** how much does the fitted edge weight move when resampling reference-labeled lag pairs across bags? For a surviving edge (i, j) , each bag produces a real fitted weight $\hat{A}_{ij}^{(b)}$ on a different random subset of reference-labeled lag pairs. The green dots show eight such bag-level estimates; the dashed horizontal line is their mean $\bar{A}_{ij}$, and the orange bracket is the bootstrap standard deviation $\widehat{\text{SD}}_{ij}$. This spread measures *estimation variability*: how reliably the edge weight is estimated under pair resampling. It does not describe how large the edge looks under scrambled data; that role belongs to the null spread s_j^{null} and effect size z_{ij} (Figure 7).

Which bags to average over. A bag in which edge (i, j) was pruned to exactly zero by L_1 contributes a structural zero, not a noisy nonzero. Two summaries are therefore reported. The *selection-conditional* estimate averages only over bags where the edge survived ($\mathcal{B}_{ij} = \{b : \text{sel}_{ij}^{(b)} = 1\}$); it reflects the typical magnitude when the edge is detected but is biased upward in magnitude. The *all-bag* estimate averages over every bag ($\mathcal{B}_{ij} = \{1, \dots, N_{\text{bag}}\}$, structural zeros included); it is diluted toward zero in proportion to how often the edge is absent. We recommend reporting the selection-conditional mean and bootstrap SD *together with* the selection frequency π_{ij} , so that the dilution is visible rather than hidden: π_{ij} and $\bar{A}_{ij}$ together convey both how often and how strongly an edge appears.

Honest caveat on calibration. The bags are overlapping random subsets from one finite reference window, not independent datasets. Shared reference state labels and repeated use of many of the same lag pairs make bags more alike than independent draws, so the naive across-bag SD can *underestimate* the true sampling error; with only $N_{\text{bag}} = 5\text{--}10$ bags it is additionally coarse (few degrees of freedom). Accordingly $\widehat{\text{SD}}_{ij}$ should be used as a *relative* reliability measure — excellent for ranking edges and for flagging unstable ones — rather than as a calibrated frequentist standard error supporting a strict confidence interval. The degree of underestimation can be probed empirically by changing `--bag_frac` and the number of bags.

8.3 Reported per-edge record

For every edge that passes stability selection ($\pi_{ij} \geq \pi_{\text{thr}}$) the pipeline emits

$$(i, j, \pi_{ij}, \bar{A}_{ij}, \widehat{\text{SD}}_{ij}, z_{ij}),$$

together with the all-bag mean for comparison and the source null summary ($m_j^{\text{null}}, s_j^{\text{null}}$). The selection frequency π_{ij} , the bagged point estimate, the bootstrap SD, and the null-referenced effect size are complementary: an edge that is frequently selected, large relative to its bootstrap SD, and far into the null tail is trustworthy; an edge strong in one bag but rarely selected, or with bootstrap SD comparable to its magnitude, is flagged as unreliable.

9 Implementation and Runtime Arguments

This section records the concrete command-line interface used by the current implementation. Stage (a) produces one bag file per invocation. Stage (b) reads the ordered set of bag files and writes a single aggregate `.prismEM.npz` file that can be opened by the existing `prismEM.eval3c.py` machinery.

9.1 `prism_FDR_Bags_train3c.py`: Stage (a)

This program is run once per bag, normally as one multi-GPU job:

```
torchrun --standalone --nnodes=1 --nproc_per_node=4 ./prism_FDR_Bags_train3c.py ...
```

It loads the reference EM output from `prismFit`, loads the original spikes from `spikesData`, samples a fixed-size subset of reference-labeled lag pairs, performs one real locked-M-step fit on that subset, runs the requested circular-shift locked-M-step null refits, and writes one self-contained bag file.

Input and output naming. The reference EM fit is

```
<basePath>/prismFit/<emFitName>.prismEM.npz.
```

The input spike file is derived from this reference fit’s provenance metadata, so Stage (a) cannot accidentally combine an EM fit from one dataset with spikes from another. For simulations this provenance

points to `state_transition_file`; for experiments it points to `experiment_name`. The corresponding spike file is read from

`<basePath>/spikesData/<sourceSpikeName>.spikes.npz`.

The output directory is `fdr_out_dir` if supplied, otherwise

`<basePath>/prismFDR`.

The output file is deterministic in `outFitName` and `bag_idx`:

`<basePath>/prismFDR/<outFitName>.bag003.prismFDRbag.npz`

for `bag_idx = 3` and default output directory. The bag index is not only a label: it also enters the random seed stream, so rerunning the same command with the same `bag_idx` reproduces the same bag and scramble shifts, while changing only `bag_idx` produces a different bag file. There is no special chronological bag; `bag_idx=0` is the first random pair subset.

A single Stage (a) bag can be evaluated directly by passing the bag stem to `prism_EM_eval3c.py`; if `dataName` contains a `bagNNN` token, the evaluator reads `<basePath>/prismFDR/<dataName>.prismFDRbag.npz` instead of looking under `prismFit`.

Required arguments.

`--basePath` Root directory containing `spikesData/` and receiving `prismFDR/`. This is the same run directory used by the plain PRISM-EM trainer.

`--emFitName` EM-train input fit stem in `prismFit/`. This file supplies the hard Viterbi state labels used by the locked M-step and the provenance record used to locate the original spike file.

`--outFitName` Stage (a) output stem in `prismFDR/`. The program writes `<outFitName>.bag<bag_idx>.prismFDRbag`. If omitted, the output stem defaults to `<emFitName>_<hash4>` with a random four-character hexadecimal suffix.

`--bag_idx` Integer bag index. It is used in the output filename and in the per-bag RNG seed. All bag indices, including `bag_idx=0`, draw random pair subsets.

Bagging and scramble arguments.

`--fdr_out_dir` Optional output directory for Stage (a) bag files. Default: `<basePath>/prismFDR`.

`--bag_frac` Fraction f_{bag} of reference-window lag pairs selected without replacement for each bag. Default: 0.8.

`--num_scrambles` Number P of circular-shift null refits saved in this bag file. Default: 4.

`--min_roll_shift_sec` Two-sided exclusion zone around zero circular shift for each neuron. A shift is drawn from the interval excluding shifts smaller than this value and shifts closer than this value to a full wrap. Default: 2 seconds.

`--null_A_init` Fresh A initialization mode for every null refit. Values: `scrambled_data` (default) or `rand`. The real locked-fit $\hat{A}$ is deliberately not used to seed the null.

`--null_B_init` Fresh B initialization mode for every null refit. Values: `scrambled_data` (default) or `rand`. The null B is refit and saved as `B_null`.

Inherited locked-M-step arguments. The Stage (a) driver shares the M-step machinery with `prism_EM_train3c.py`, but it does not run a bag-specific E-step. The default values are loaded from the reference EM metadata so a bag job is reproducible from the reference fit alone. Any argument listed here may be redefined on the `prism_FDR_Bags_train3c.py` command line; the resolved values are saved in the bag metadata and used identically for the real locked fit and all null locked fits in that bag. This lets the reference EM run be tuned for state assignment while the bag jobs are tuned for conditional $A, \mathbf{B}$ estimation.

- `--m_epochs` M-step Adam epochs for each locked fit. The null locked M-step uses the same count; there is no separate `null_m_epochs`.
- `--lr_mstep` Adam learning rate for the M-step. Default: 0.003.
- `--lr_end_factor` Final learning-rate multiplier after linear decay. Default: 0.1.
- `--lambda3` L1 penalty weight on off-diagonal A . Default: 0.02.
- `--rho_max` Target spectral radius for soft correction of A . Default: 0.95.
- `--prescale_m_step_4_ArhoMax` M-step batch interval for spectral-radius projection checks. Default: 120.
- `--delay_em_iter_4_ArhoMax`, `--delay_em_iter_4_lrDecay`, `--delay_em_iter_4_Aprune` Accepted by the shared M-step configuration. For locked FDR refits, schedule-gated operations are active from the first locked epoch unless the implementation explicitly maps these controls to locked-epoch delays.
- `--batch_size` M-step minibatch size. Default: 2048.
- `--init_A` Initial A mode for the locked real fit: `data` (default), `ref`, or `rand`.
- `--init_B` Initial $\mathbf{B}$ mode for the locked real fit: `data` (default), `ref`, or `rand`. The default `data` mode is chosen to capture bag-to-bag sampling variance in the FDR procedure. Initializing every real bag from the same reference EM $\hat{A}, \hat{\mathbf{B}}$ is faster and stable, but it can make bags overly correlated with the reference solution and understate the variability that the stability filter is meant to measure.
- `--init_A_Tmax` Maximum number of bins used for data-driven A initialization. Default: 50000.
- `--seed` Base random seed. Default: 42; the effective seed is offset by `bag_idx`.
- `--verb` Verbosity level. With the default verbosity, rank 0 reports runtime device information, real-fit elapsed time, and elapsed time for each scramble refit.

The bag trainer does not expose state-discovery controls such as `--num_em_iters`, `--pgd_iter`, `--lr_estep`, `--lambda2`, `--init_states`, `--decode_dwell_sec`, or `--num_states`. These belong to the reference `prism_EM_train3c.py` run. In particular, `--num_states` is inferred from the locked reference labels.

Saved Stage (a) contents. The file has the real locked-fit fields ($\hat{A}$, $\hat{\mathbf{B}}$) plus reference-derived time-series fields needed for plotting ($\hat{S}$, $\hat{c}$, and confidence/loss summaries when saved upstream). It also contains $\hat{A}_{\text{null}}$, $\hat{\mathbf{B}}_{\text{null}}$, `roll_shifts_bin`, `bag_pair_indices`, `bag_frac`, reference fit provenance, and locked M-step histories. Stage (a) metadata are grouped under `bagsFDR_stageA`. This includes the pair-sampling controls, the circular-shift null-refit controls, per-scramble initialization diagnostics, and a reference-fit sub-dict recording the `prismFit` file and time range used to make the locked state labels. It also includes the effective locked-M-step hyperparameters after applying command-line overrides to the metadata defaults.

big_fit_bags.sh launch pattern. The helper script `big_fit_bags.sh` is a concrete end-to-end launch example for one dataset. It sets

```
basePath=/pscratch/sd/b/balewski/2026_causalNet_exp_ver3c
```

and a short input spike name such as `daleN200_102308_a5d24b`. The script derives

```
runTag=<hash4>,      emFitName=<shortN>_em<runTag>
```

as a random four-character hexadecimal tag. It first runs `prism_EM_train3c.py` on the configured `timeRange` and writes

```
<basePath>/prismFit/<emFitName>.prismEM.npz.
```

Then all bags launched inside the same script use the same Stage (a) extended name

```
<shortN>_em<runTag>_fdr<runTag>.
```

For each requested bag index, the script runs `torchrun --standalone --nnodes=1 --nproc_per_node=4` on `prism_FDR_Bags_train3c.py` with `--emFitName <emFitName>` and `--outFitName <shortN>_em<runTag>_fdr<runTag>`. With `numBags=6`, the expected Stage (a) output files are

```
<basePath>/prismFDR/<shortN>_em<runTag>_fdr<runTag>.bag000.prismFDRbag.npz
<basePath>/prismFDR/<shortN>_em<runTag>_fdr<runTag>.bag001.prismFDRbag.npz
...
<basePath>/prismFDR/<shortN>_em<runTag>_fdr<runTag>.bag005.prismFDRbag.npz
```

These are all equivalent random locked-M-step bags; `bag000` is not a chronological reference. If aggregation is enabled, Stage (b) is called with `--dataName <shortN>_em<runTag>_fdr<runTag>`, `--outAgrName <shortN>_em<runTag>_fdr<runTag>_agr<runTag>`, and `--num_bags` equal to the number of contiguous bags starting from `bag000` that are present on disk.

9.2 prism_EM_FDR_Bags_aggregate3c.py: Stage (b)

This program is a single CPU postprocessing step. It reads the Stage (a) files, performs the per-source null thresholding and cross-bag stability selection, and writes one aggregate file in the same outer format expected by `prism_EM_eval3c.py`.

Input and output naming. The default input directory is

```
<basePath>/prismFDR.
```

For `num_bags = B`, the program expects the contiguous file set

```
<dataName>.bag000.prismFDRbag.npz
<dataName>.bag001.prismFDRbag.npz
...
<dataName>.bag{num_bags-1, zero padded}.prismFDRbag.npz
```

Here `dataName` is the Stage (a) output stem, i.e. the same value passed as `--outFitName` to `prism_FDR_Bags_train3c.py`. The output stem is set explicitly by `--outAgrName`; this makes it easy to rerun only the aggregation with different thresholds while preserving the same input bag stem. If `--outAgrName` is omitted, the output stem defaults to `<dataName>_<hash4>` with a random four-character hexadecimal suffix. The output file is written to

```
<basePath>/prismFit/<outAgrName>.prismEM.npz.
```


For example, with `dataName=daleN100_2ba29b_c47b43_em1234_fdr1234`, one possible output stem is

`daleN100_2ba29b_c47b43_em1234_fdr1234_agrA.`

It can then be opened by

```
./prism_EM_eval3c.py \  
  --basePath <basePath> \  
  --dataName <outAgrName> \  
  -p a e f g
```

Runtime arguments.

- `--basePath` Root run directory. The default input is `<basePath>/prismFDR`; the aggregate output is written to `<basePath>/prismFit`.
- `--dataName` Extended Stage (a) output data name, normally the Stage (a) output stem. This exact string is the prefix of all bag files consumed by Stage (b), e.g. `<dataName>.bag000.prismFDRbag.npz`.
- `--outAgrName` Aggregate output stem written to `prismFit/<outAgrName>.prismEM.npz`. This is intentionally separate from `--dataName` so repeated aggregation choices can create differently named outputs from the same bag files. If omitted, the program appends a random `<hash4>` suffix to `dataName`.
- `--num_bags` Number of Stage (a) bag files to read. The implementation expects bags numbered contiguously from `bag000`.
- `--per_bag_quantile` Quantile q used for the per-source null magnitude threshold $\tau_j^{(b)}$. Default: 0.99.
- `--stab_sel_thresh` Cross-bag stability selection-frequency threshold π_{thr} . Default: 0.7.
- `--fdr_out_dir` Optional override for the Stage (a) bag directory. Default: `<basePath>/prismFDR`.
- `--verb` Verbosity level. Default: 1.

Aggregate output schema. The output `A_hat` is the final stable matrix: off-diagonal entries are the selection-conditional bagged means for edges with $\pi_{ij} \geq \text{stab_sel_thresh}$, and zero otherwise. With the default `--stab_sel_thresh 0.7`, this means retaining only edges present in at least 70% of bags; the retained edge value is the unweighted mean over only those bag instances where the edge was selected. Diagonal entries are the all-bag mean diagonals from the real bag fits; the corresponding diagonal mean, standard deviation, and standard error are also saved as `A_diag_mean`, `A_diag_std`, and `A_diag_stderr`. `A_prune`, `neuron_type`, and `neuron_Sedge` are recomputed from this aggregate `A_hat` using source columns. First `min_posW` and `max_negW` are computed from the stable off-diagonal `A_hat` weights. Then `neuron_Sedge` is the off-diagonal column sum for each source neuron, and source columns are classified as excitatory when `neuron_Sedge > min_posW`, inhibitory when `neuron_Sedge < max_negW`, and undecided otherwise. `A_prune` preserves the diagonal, keeps only positive outgoing weights for excitatory source columns, keeps only negative outgoing weights for inhibitory source columns, and leaves undecided source columns sign-unpruned. The output `B_hat` is the mean across bags in the reference state order. Because every bag uses the hard labels from the same reference EM fit, state rows are already aligned; no bag-to-bag state permutation is needed other than an optional sanity check. The time-series fields `S_hat`, `c_hat`, `S_hat_CL`, and the EM histories are copied from the reference EM output to preserve compatibility with the existing plotting code; those plots should therefore be interpreted as reference-fit diagnostics, while the connectivity fields are Stage (b) aggregates. Stage (b) metadata are

grouped under `bagsFDR_stageB`; reference provenance and sampled-pair metadata from Stage (a) remain under `bagsFDR_stageA`.

Additional Stage (b) arrays include `A_bag`, `B_bag`, `selected_mask`, `selected_in_bag`, `selection_count`, `selection_frequency`, `A_mean_selected`, `A_sd_selected`, `A_mean_all`, `A_sd_all`, `A_diag_mean`, `A_diag_std`, `A_diag_stderr`, `src_null_tau_bag`, `src_null_tau_mean`, `src_null_mean`, `src_null_sd`, `z_null`, and a compact selected-edge table with `edge_i`, `edge_j`, `edge_sel_freq`, `edge_A_mean`, `edge_A_sd_boot`, and `edge_z_null`, plus `edge_src_null_mean`, `edge_src_null_sd`, and `edge_src_tau_mean`.

10 End-to-End Summary

Reference EM fit, run once.

1. Run `prism_EM_train3c.py` on the desired `--time_range_sec`; save `prismFit/<emFitName>.prismEM.npz`, for example `prismFit/<dataName>_emjXXXX.prismEM.npz`.
2. Use its saved reference window, $\hat{S}^{\text{ref}}$, and $\hat{c}^{\text{ref}}$ as the only state timeline for all FDR bags.

Stage (a), run N_{bag} times in parallel (one node, $4 \times \mathbf{A100}$ each).

1. Load `prismFit/<emFitName>.prismEM.npz` and recover the input spike stem from its provenance metadata. Then load `spikesData/<sourceSpikeName>.spikes.npz` and form the reference-labeled pair table $(H_t, X_t, Z_t) = (\hat{S}_t^{\text{ref}}, Y_{t-1}, Y_t)$.
2. Draw $K_{\text{bag}} = \lfloor \text{bag_frac}(T_{\text{ref}} - 1) \rfloor$ pair indices without replacement, using the same selected rows for all neurons.
3. Run `mstep.locked` $\rightarrow (\hat{A}^{(b)}, \hat{\mathbf{B}}^{(b)})$ on the selected real rows with $H_t = \hat{S}_t^{\text{ref}}$ fixed.
4. For $p = 1, \dots, P$: draw independent per-neuron circular shifts in the two-sided zone $[\delta_{\text{roll}}^{\text{min,bin}}, T_{\text{ref}} - \delta_{\text{roll}}^{\text{min,bin}}]$, roll each neuron's reference-window spikes, form scrambled pair rows using the same selected indices, and `mstep.locked` $\rightarrow (A^{\text{null},(b,p)}, \mathbf{B}^{\text{null},(b,p)})$ with the locked labels $H_t = \hat{S}_t^{\text{ref}}$, fresh $\mathbf{A}$ initialization controlled by `null_A_init` $\in \{\text{scrambled_data}, \text{rand}\}$, and fresh $\mathbf{B}$ initialization controlled by `null_B_init` $\in \{\text{scrambled_data}, \text{rand}\}$.
5. Write one file for bag b , named from `dataName` and `bag_idx`, containing $\hat{A}^{(b)}$, $\{A^{\text{null},(b,p)}\}$, $\{\mathbf{B}^{\text{null},(b,p)}\}$, sampled pair indices, realized shifts, fitted sparsity, reference-fit provenance, and history summary.

Stage (b), single postprocessing job.

1. Per bag and source column, pool the $(N-1)P$ off-diagonal null weights, form the upper- q magnitude threshold $\tau_j^{(b)}$, and select edges with $|\hat{A}_{ij}^{(b)}| > \tau_j^{(b)}$; record q_{Λ} .
2. Compute selection frequencies π_{ij} and keep edges with $\pi_{ij} \geq \pi_{\text{thr}}$, where π_{thr} is supplied by `--stab_sel_thresh`.
3. For surviving edges, compute the bagged mean $\bar{A}_{ij}$, bootstrap SD $\widehat{\text{SD}}_{ij}$ (selection-conditional and all-bag), and effect size z_{ij} against the pooled source null.
4. Emit the vetted edge list $(i, j, \pi_{ij}, \bar{A}_{ij}, \widehat{\text{SD}}_{ij}, z_{ij})$ plus diagnostics.

Closing note. The method’s robustness comes from separating state discovery from edge testing: connectivity is asserted only when it survives repeated random subsampling of reference-labeled lag pairs, exceeds a null built by the identical locked-M-step machinery (scramble calibration), and recurs across bags (stability selection). Its two reported uncertainties — the null spread and the across-bag bootstrap SD — are deliberately kept distinct, and their known limitations under overlapping pair subsets are stated rather than papered over, so that the resulting edge list can be read with appropriate confidence on real data where no ground truth is available to check it.